Journal of Machine Learning Research 23 (2022) 1-40 Submitted 8/21; Revised 3/22; Published 4/22

arXiv:2101.03961v3 [cs.LG] 16 Jun 2022

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity

William Fedus*
LIAMFEDUS@GOOGLE.COM

Barret Zoph*
BARRETZOPH@GOOGLE.COM

Noam Shazeer
NOAM@GOOGLE.COM
Google, Mountain View, CA 94043, USA

Editor: Alexander Clark

Abstract

In deep learning, models typically reuse the same parameters for all inputs. Mixture of Experts (MoE) models defy this and instead select *different* parameters for each incoming example. The result is a sparsely-activated model—with an outrageous number of parameters—but a constant computational cost. However, despite several notable successes of MoE, widespread adoption has been hindered by complexity, communication costs, and training instability. We address these with the introduction of the Switch Transformer. We simplify the MoE routing algorithm and design intuitive improved models with reduced communication and computational costs. Our proposed training techniques mitigate the instabilities, and we show large sparse models may be trained, for the first time, with lower precision (bfloat16) formats. We design models based off T5-Base and T5-Large (Raffel et al., 2019) to obtain up to 7x increases in pre-training speed with the same computational resources. These improvements extend into multilingual settings where we measure gains over the mT5-Base version across all 101 languages. Finally, we advance the current scale of language models by pre-training up to trillion parameter models on the "Colossal Clean Crawled Corpus", and achieve a 4x speedup over the T5-XXL model.[1][2]

Keywords: mixture-of-experts, natural language processing, sparsity, large-scale machine learning, distributed computing

∗. Equal contribution.

1. JAX code for Switch Transformer and all model checkpoints are available at `https://github.com/google-research/t5x`
2. Tensorflow code for Switch Transformer is available at `https://github.com/tensorflow/mesh/blob/master/mesh_tensorflow/transformer/moe.py`

©2022 William Fedus, Barret Zoph and Noam Shazeer.

License: CC-BY 4.0, see `https://creativecommons.org/licenses/by/4.0/`. Attribution requirements are provided at `http://jmlr.org/papers/v23/21-0998.html`.

Contents

1 Introduction **3**

2 Switch Transformer **4**
2.1 Simplifying Sparse Routing 5
2.2 Efficient Sparse Routing 6
2.3 Putting It All Together: The Switch Transformer 8
2.4 Improved Training and Fine-Tuning Techniques 8

3 Scaling Properties **11**
3.1 Scaling Results on a Step-Basis 12
3.2 Scaling Results on a Time-Basis 13
3.3 Scaling Versus a Larger Dense Model 13

4 Downstream Results **14**
4.1 Fine-Tuning 14
4.2 Distillation 16
4.3 Multilingual Learning 17

5 Designing Models with Data, Model, and Expert-Parallelism **18**
5.1 Data Parallelism 20
5.2 Model Parallelism 20
5.3 Model and Data Parallelism 21
5.4 Expert and Data Parallelism 22
5.5 Expert, Model and Data Parallelism 22
5.6 Towards Trillion Parameter Models 22

6 Related Work **24**

7 Discussion **25**

8 Future Work **26**

9 Conclusion **27**

A Switch for Attention **27**

B Preventing Token Dropping with *No-Token-Left-Behind* **29**

C Encouraging Exploration Across Experts **29**

D Switch Transformers in Lower Compute Regimes **29**

E Relation of Upstream to Downstream Model Performance **32**

F Pseudo Code for Switch Transformers **33**

1. Introduction

Large scale training has been an effective path towards flexible and powerful neural language models (Radford et al., 2018; Kaplan et al., 2020; Brown et al., 2020). Simple architectures—backed by a generous computational budget, data set size and parameter count—surpass more complicated algorithms (Sutton, 2019). An approach followed in Radford et al. (2018); Raffel et al. (2019); Brown et al. (2020) expands the model size of a densely-activated Transformer (Vaswani et al., 2017). While effective, it is also extremely computationally intensive (Strubell et al., 2019). Inspired by the success of model scale, but seeking greater computational efficiency, we instead propose a *sparsely-activated* expert model: the Switch Transformer. In our case the sparsity comes from activating a *subset* of the neural network weights for each incoming example.

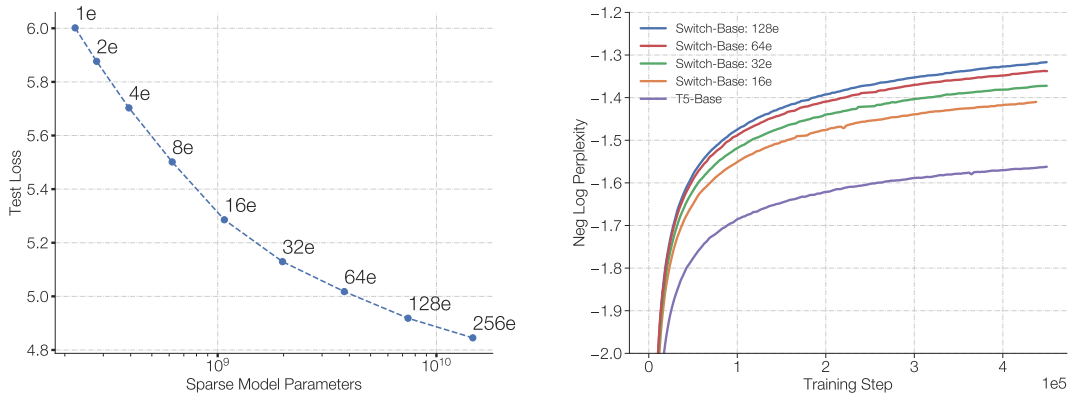

Figure 1: Scaling and sample efficiency of Switch Transformers. Left Plot: Scaling properties for increasingly sparse (more experts) Switch Transformers. Right Plot: Negative log perplexity comparing Switch Transformers to T5 (Raffel et al., 2019) models using the same compute budget.

Sparse training is an active area of research and engineering (Gray et al., 2017; Gale et al., 2020), but as of today, machine learning libraries and hardware accelerators still cater to dense matrix multiplications. To have an efficient sparse algorithm, we start with the Mixture-of-Expert (MoE) paradigm (Jacobs et al., 1991; Jordan and Jacobs, 1994; Shazeer et al., 2017), and simplify it to yield training stability and computational benefits. MoE models have had notable successes in machine translation (Shazeer et al., 2017, 2018; Lepikhin et al., 2020), however, widespread adoption is hindered by complexity, communication costs, and training instabilities.

We address these issues, and then go beyond translation, to find that these class of algorithms are broadly valuable in natural language. We measure superior scaling on a diverse set of natural language tasks and across three regimes in NLP: pre-training, fine-tuning and multi-task training. While this work focuses on scale, we also show that the Switch Transformer architecture not only excels in the domain of supercomputers, but is

beneficial even with only a few computational cores. Further, our large sparse models can be distilled (Hinton et al., 2015) into small dense versions while preserving 30% of the sparse model quality gain. Our contributions are the following:

- The Switch Transformer architecture, which simplifies and improves over Mixture of Experts.
- Scaling properties and a benchmark against the strongly tuned T5 model (Raffel et al., 2019) where we measure 7x+ pre-training speedups while still using the same FLOPS per token. We further show the improvements hold even with limited computational resources, using as few as two experts.
- Successful distillation of sparse pre-trained and specialized fine-tuned models into small dense models. We reduce the model size by up to 99% while preserving 30% of the quality gains of the large sparse teacher.
- Improved pre-training and fine-tuning techniques: **(1)** selective precision training that enables training with lower bfloat16 precision **(2)** an initialization scheme that allows for scaling to a larger number of experts and **(3)** increased expert regularization that improves sparse model fine-tuning and multi-task training.
- A measurement of the pre-training benefits on multilingual data where we find a universal improvement across all 101 languages and with 91% of languages benefiting from 4x+ speedups over the mT5 baseline (Xue et al., 2020).
- An increase in the scale of neural language models achieved by efficiently combining data, model, and expert-parallelism to create models with up to a trillion parameters. These models improve the pre-training speed of a strongly tuned T5-XXL baseline by 4x.

2. Switch Transformer

The guiding design principle for Switch Transformers is to maximize the parameter count of a Transformer model (Vaswani et al., 2017) in a simple and computationally efficient way. The benefit of scale was exhaustively studied in Kaplan et al. (2020) which uncovered power-law scaling with model size, data set size and computational budget. Importantly, this work advocates training large models on relatively small amounts of data as the computationally optimal approach.

Heeding these results, we investigate a fourth axis: increase the *parameter count* while keeping the floating point operations (FLOPs) per example constant. Our hypothesis is that the parameter count, independent of total computation performed, is a separately important axis on which to scale. We achieve this by designing a sparsely activated model that efficiently uses hardware designed for dense matrix multiplications such as GPUs and TPUs. Our work here focuses on TPU architectures, but these class of models may be similarly trained on GPU clusters. In our distributed training setup, our sparsely activated layers split *unique* weights on different devices. Therefore, the weights of the model increase with the number of devices, all while maintaining a manageable memory and computational footprint on each device.

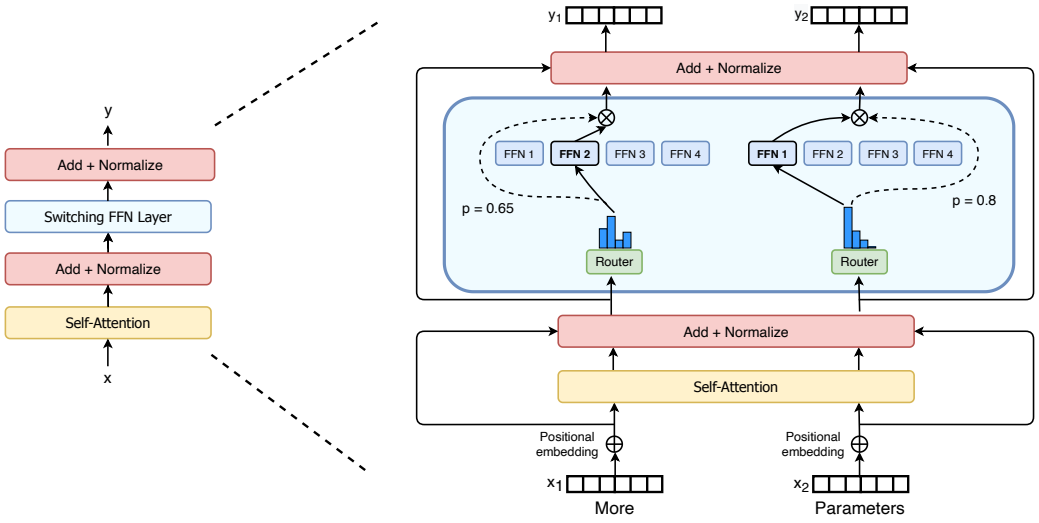

Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = "More" and x_2 = "Parameters" below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

2.1 Simplifying Sparse Routing

Mixture of Expert Routing. Shazeer et al. (2017) proposed a natural language Mixture-of-Experts (MoE) layer which takes as an input a token representation x and then routes this to the best determined top-k experts, selected from a set $\{E_i(x)\}_{i=1}^{N}$ of N experts. The router variable W_r produces logits $h(x) = W_r \cdot x$ which are normalized via a softmax distribution over the available N experts at that layer. The gate-value for expert i is given by,

$$p_i(x) = \frac{e^{h(x)_i}}{\sum_j^N e^{h(x)_j}}. \tag{1}$$

The top-k gate values are selected for routing the token x. If $\mathcal{T}$ is the set of selected top-k indices then the output computation of the layer is the linearly weighted combination of each expert's computation on the token by the gate value,

$$y = \sum_{i \in \mathcal{T}} p_i(x) E_i(x). \tag{2}$$

Switch Routing: Rethinking Mixture-of-Experts. Shazeer et al. (2017) conjectured that routing to $k > 1$ experts was necessary in order to have non-trivial gradients to the routing functions. The authors intuited that learning to route would not work without the ability to compare at least two experts. Ramachandran and Le (2018) went further to